# Lab 2 - Simple Linear Regression

**Dataset:** Department Awareness Survey (Class Survey Form Responses)

**Experiments:**

- Experiment 1: CIA % → GPA
- Experiment 2: Attendance % → GPA

In [91]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
```

## Part A: Data Collection and Preprocessing

In [92]:
```
# Step 1: Load the dataset
df = pd.read_csv('Department Awareness Survey (Responses).xlsx - Form Responses 1.csv')
df.columns = df.columns.str.strip()
print("Dataset loaded successfully!")
```

```
Dataset loaded successfully!
```

In [93]:
```
# Step 2: Display first 5 rows
df.head()
```

Out[93]:

| | Timestamp | Registration Number | Email | Job role that you are interested in | What is the minimum salary of students placed through campus (In LPA..respond as a number) | What is the maximum salary of students placed through campus (In LPA..respond as a number) | What is the median salary of students placed through campus (In LPA..respond as a number) | Which is the highest paying company | R con |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 6/22/2026 8:49:31 | 2547123 | jiyaelza.jabi@mca.christuniversity.in | Software Engineer | 4 | Option 1 | 9 | Deolite | |
| **1** | 6/22/2026 8:49:51 | 2547122 | jinishaleema.rosario@mca.christuniversity.in | Data Engineer | 7,00,000 | Option 1 | 4,50,000 | Deolite | |
| **2** | 6/22/2026 8:50:45 | 2547101 | rajeev.chandar@mca.christuniversity.in | Software Engineer | 6 | 17 | 8 | DE Shaw | |
| **3** | 6/22/2026 8:51:01 | 2547156 | sounak.chakraborty@mca.christuniversity.in | Data Scientist | 4.5 | 12 | 6.9 | DE Shaw | |
| **4** | 6/22/2026 8:51:42 | 2547148 | samar.subhash@mca.christuniversity.in | Software Engineer | 8 | 12 | 10 | EY | |

In [94]:
```
# Step 3: Check dataset dimensions
print("Number of rows   :", df.shape[0])
print("Number of columns:", df.shape[1])
print("\nThe dataset has", df.shape[0], "records and", df.shape[1], "columns.")
```

```
Number of rows   : 54
Number of columns: 15

The dataset has 54 records and 15 columns.
```

In [95]:
```
# Step 4: Identify missing values
print("Missing values in each column:")
print(df.isnull().sum())
print("\nTotal missing values:", df.isnull().sum().sum())
```

```
Missing values in each column:
Timestamp                                                                    0
Registration Number                                                          0
Email                                                                        0
Job role that you are interested in                                          0
What is the minimum salary of students placed through campus (In LPA..respond as a number)    0
What is the maximum salary of students placed through campus (In LPA..respond as a number)    0
What is the  median salary of students placed through campus (In LPA..respond as a number)    0
Which is the highest paying company                                          0
Rate your contribution towards extra curricular activities                   0
Rate your technical competencies                                             0
What are your package expectations (LPA)                                     0
your CIA % of last semester                                                  0
your GPA of last semester                                                    0
Your maximum attendance % till last semester                                 0
Internships Interests                                                        0
dtype: int64

Total missing values: 0
```

In [96]:
```
# Step 5: Remove null values
before = len(df)
df = df.dropna()
after = len(df)
print("Rows before removing nulls:", before)
print("Rows after  removing nulls:", after)
print("Rows removed:", before - after)
```

```
Rows before removing nulls: 54
Rows after  removing nulls: 54
Rows removed: 0
```

In [97]:
```
# Step 6: Convert required columns to numerical datatype
# Rename columns for easy access
df = df.rename(columns={
    'your CIA % of last semester'              : 'CIA',
    'your GPA of last semester'                : 'GPA',
    'Your maximum attendance % till last semester': 'Attendance'
})

# Convert to numbers (any invalid text like 'Option 1' becomes NaN)
```

```
df['CIA']        = pd.to_numeric(df['CIA'],        errors='coerce')
df['GPA']        = pd.to_numeric(df['GPA'],        errors='coerce')
df['Attendance'] = pd.to_numeric(df['Attendance'], errors='coerce')

print("Column data types after conversion:")
print(df[['CIA', 'GPA', 'Attendance']].dtypes)
print("\nValid (non-null) values:")
print("CIA       :", df['CIA'].notna().sum())
print("GPA       :", df['GPA'].notna().sum())
print("Attendance:", df['Attendance'].notna().sum())
```

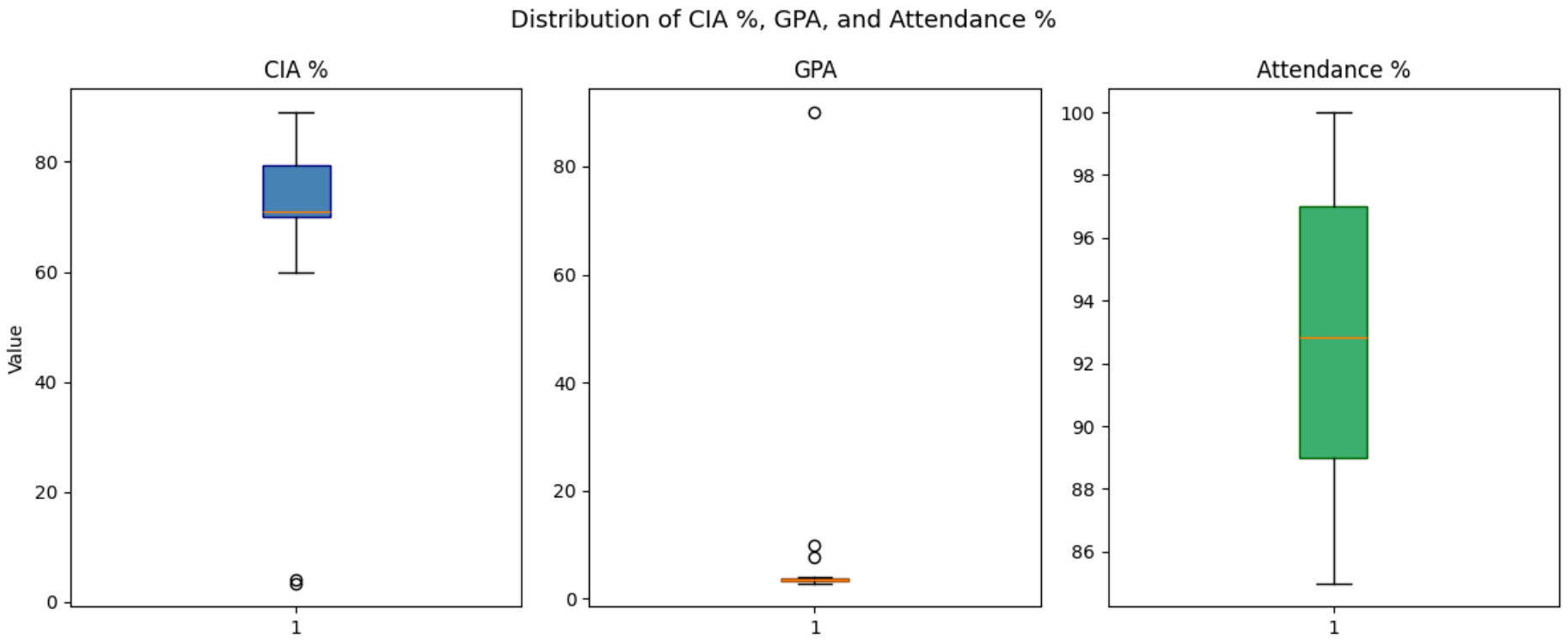
```
Column data types after conversion:
CIA           float64
GPA           float64
Attendance    float64
dtype: object

Valid (non-null) values:
CIA       : 44
GPA       : 53
Attendance: 41
```

In [98]:
```
# Step 7: Remove duplicate records
before = len(df)
df = df.drop_duplicates()
after = len(df)
print("Rows before removing duplicates:", before)
print("Rows after  removing duplicates:", after)
print("Duplicate rows removed:", before - after)
```

```
Rows before removing duplicates: 54
Rows after  removing duplicates: 54
Duplicate rows removed: 0
```

In [99]:
```
# Step 8: Generate statistical summary using box plots
fig, axes = plt.subplots(1, 3, figsize=(12, 5))

axes[0].boxplot(df['CIA'].dropna(), patch_artist=True,
                boxprops=dict(facecolor='steelblue', color='navy'))
axes[0].set_title('CIA %')
axes[0].set_ylabel('Value')
```

```
axes[1].boxplot(df['GPA'].dropna(), patch_artist=True,
                boxprops=dict(facecolor='darkorange', color='saddlebrown'))
axes[1].set_title('GPA')

axes[2].boxplot(df['Attendance'].dropna(), patch_artist=True,
                boxprops=dict(facecolor='mediumseagreen', color='darkgreen'))
axes[2].set_title('Attendance %')

plt.suptitle('Distribution of CIA %, GPA, and Attendance %', fontsize=13)
plt.tight_layout()
plt.show()

print("Reading the box plot:")
print("- Middle line  : Median (middle value)")
print("- Box edges    : 25th and 75th percentile (middle 50% of data)")
print("- Whiskers     : Min and Max (within normal range)")
print("- Dots outside : Outliers")
```




```
Reading the box plot:
- Middle line  : Median (middle value)
- Box edges    : 25th and 75th percentile (middle 50% of data)
- Whiskers     : Min and Max (within normal range)
- Dots outside : Outliers
```

In [100…

```python
# Step 9: Select dependent and independent variables
print("Experiment 1:")
print("  Independent Variable (X): CIA Percentage")
print("  Dependent Variable   (Y): GPA")
print()
print("Experiment 2:")
print("  Independent Variable (X): Attendance Percentage")
print("  Dependent Variable   (Y): GPA")
```

```
Experiment 1:
  Independent Variable (X): CIA Percentage
  Dependent Variable   (Y): GPA

Experiment 2:
  Independent Variable (X): Attendance Percentage
  Dependent Variable   (Y): GPA
```

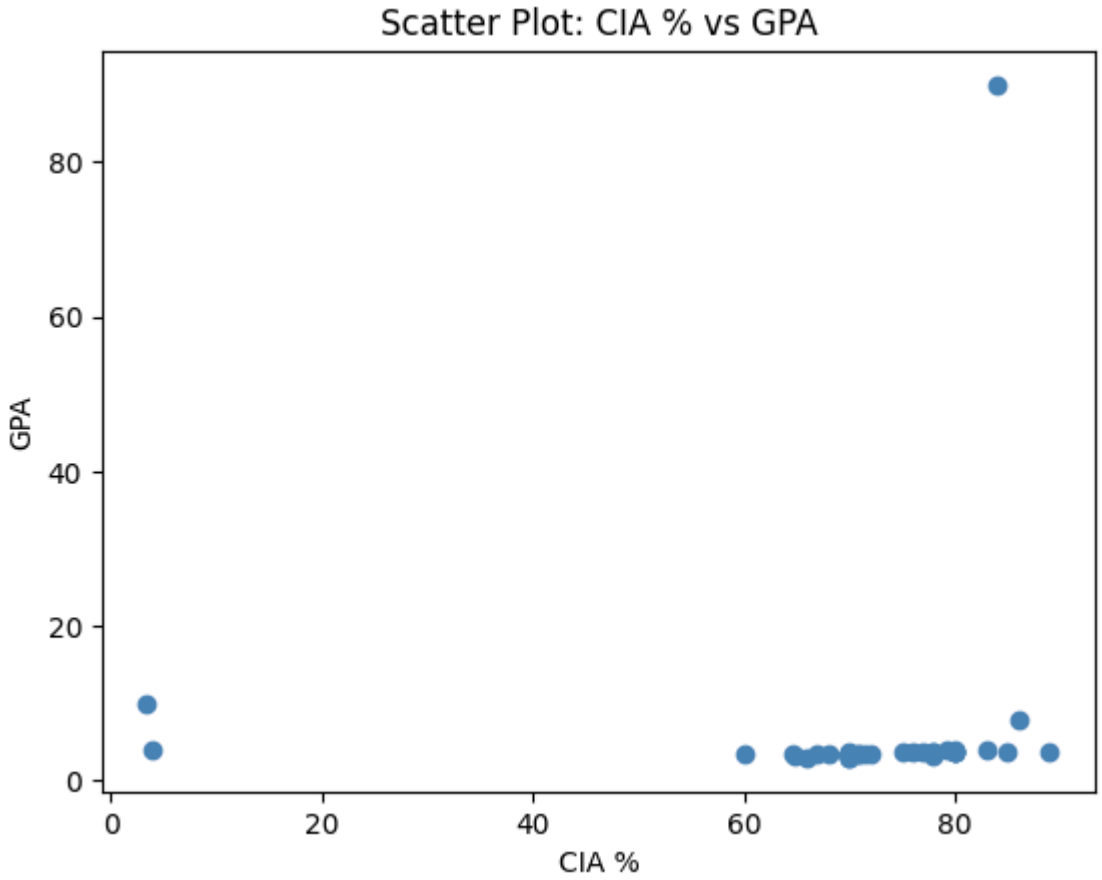
---

# Experiment 1: CIA Percentage → GPA

In [101…

```python
# Prepare data - drop rows where CIA or GPA is missing
data1 = df[['CIA', 'GPA']].dropna()
print("Number of data points used:", len(data1))

# Scatter plot to see the relationship
plt.scatter(data1['CIA'], data1['GPA'], color='steelblue')
plt.xlabel('CIA %')
plt.ylabel('GPA')
plt.title('Scatter Plot: CIA % vs GPA')
plt.show()

print("Each dot represents one student.")
print("We can see if higher CIA % tends to go with higher GPA.")
```

```
Number of data points used: 43
```




Each dot represents one student.
We can see if higher CIA % tends to go with higher GPA.

In [102…

```
# Fit Simple Linear Regression model
X1 = data1[['CIA']]   # independent variable (2D array needed by sklearn)
y1 = data1['GPA']     # dependent variable

model1 = LinearRegression()
model1.fit(X1, y1)

# Predict GPA using the model
y1_pred = model1.predict(X1)

# Results
```

```
slope1     = model1.coef_[0]
intercept1 = model1.intercept_
r2_1       = r2_score(y1, y1_pred)
mse1       = mean_squared_error(y1, y1_pred)

print("=== Experiment 1 Results ===")
print(f"Slope     : {slope1:.4f}")
print(f"Intercept : {intercept1:.4f}")
print(f"R2 Score  : {r2_1:.4f}")
print(f"MSE       : {mse1:.4f}")
print()
print("Interpretation:")
print(f"- Slope {slope1:.4f}: For every 1% increase in CIA, GPA increases by {slope1:.4f} points.")
print(f"- Intercept {intercept1:.4f}: This is the predicted GPA when CIA is 0 (baseline value).")
print(f"- R2 Score {r2_1:.4f}: The model explains {r2_1*100:.2f}% of the variation in GPA.")
print(f"- MSE {mse1:.4f}: The average squared difference between actual and predicted GPA is {mse1:.4f}.")
```

```
=== Experiment 1 Results ===
Slope     : 0.0700
Intercept : 0.7864
R2 Score  : 0.0074
MSE       : 169.2125

Interpretation:
- Slope 0.0700: For every 1% increase in CIA, GPA increases by 0.0700 points.
- Intercept 0.7864: This is the predicted GPA when CIA is 0 (baseline value).
- R2 Score 0.0074: The model explains 0.74% of the variation in GPA.
- MSE 169.2125: The average squared difference between actual and predicted GPA is 169.2125.
```

In [103…

```
# Plot regression line over scatter plot
plt.scatter(data1['CIA'], y1, color='steelblue', label='Actual Data')
plt.plot(data1['CIA'], y1_pred, color='red', label='Regression Line')
plt.xlabel('CIA %')
plt.ylabel('GPA')
plt.title('Experiment 1: CIA % vs GPA with Regression Line')
plt.legend()
plt.show()
```


---

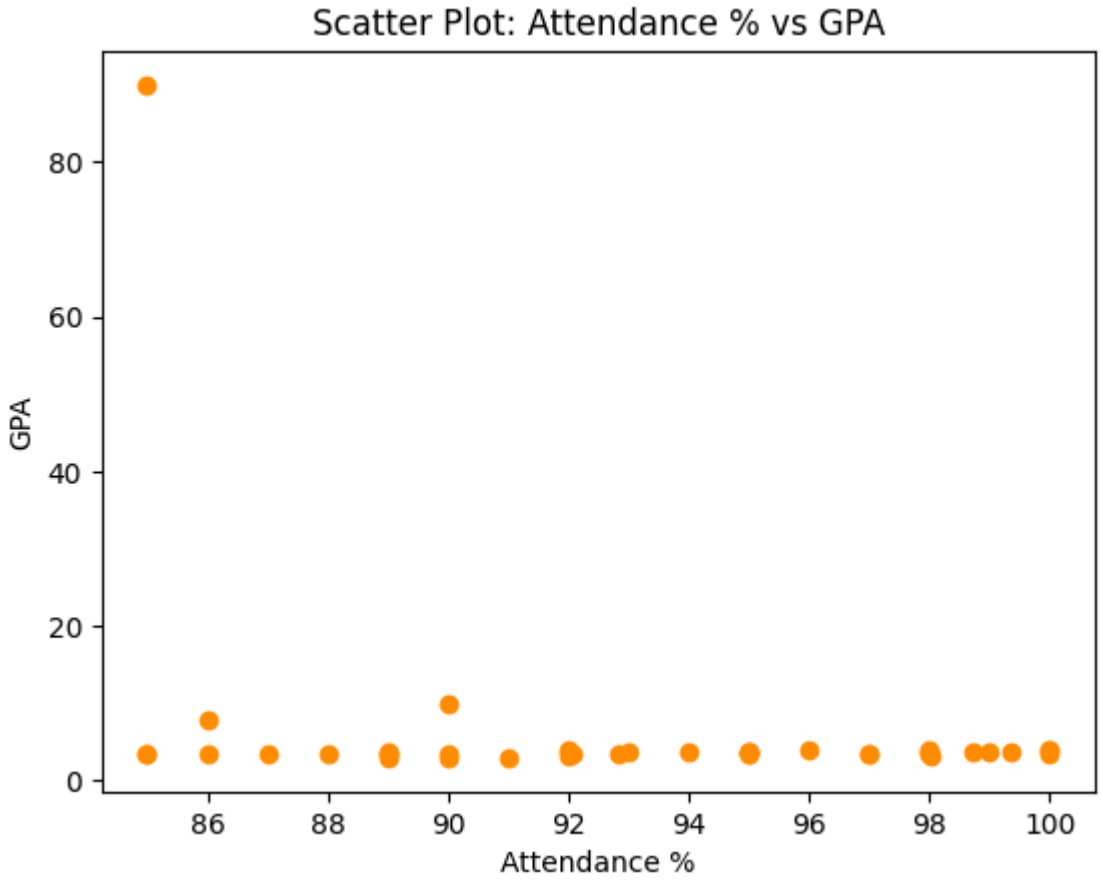
## Experiment 2: Attendance Percentage → GPA

In [104…

```
# Prepare data - drop rows where Attendance or GPA is missing
data2 = df[['Attendance', 'GPA']].dropna()
print("Number of data points used:", len(data2))

# Scatter plot to see the relationship
plt.scatter(data2['Attendance'], data2['GPA'], color='darkorange')
plt.xlabel('Attendance %')
```

```
plt.ylabel('GPA')
plt.title('Scatter Plot: Attendance % vs GPA')
plt.show()

print("Each dot represents one student.")
print("We can see if higher Attendance % tends to go with higher GPA.")
```

```
Number of data points used: 40
```




```
Each dot represents one student.
We can see if higher Attendance % tends to go with higher GPA.
```

In [105…

```
# Fit Simple Linear Regression model
X2 = data2[['Attendance']]  # independent variable
y2 = data2['GPA']           # dependent variable
```

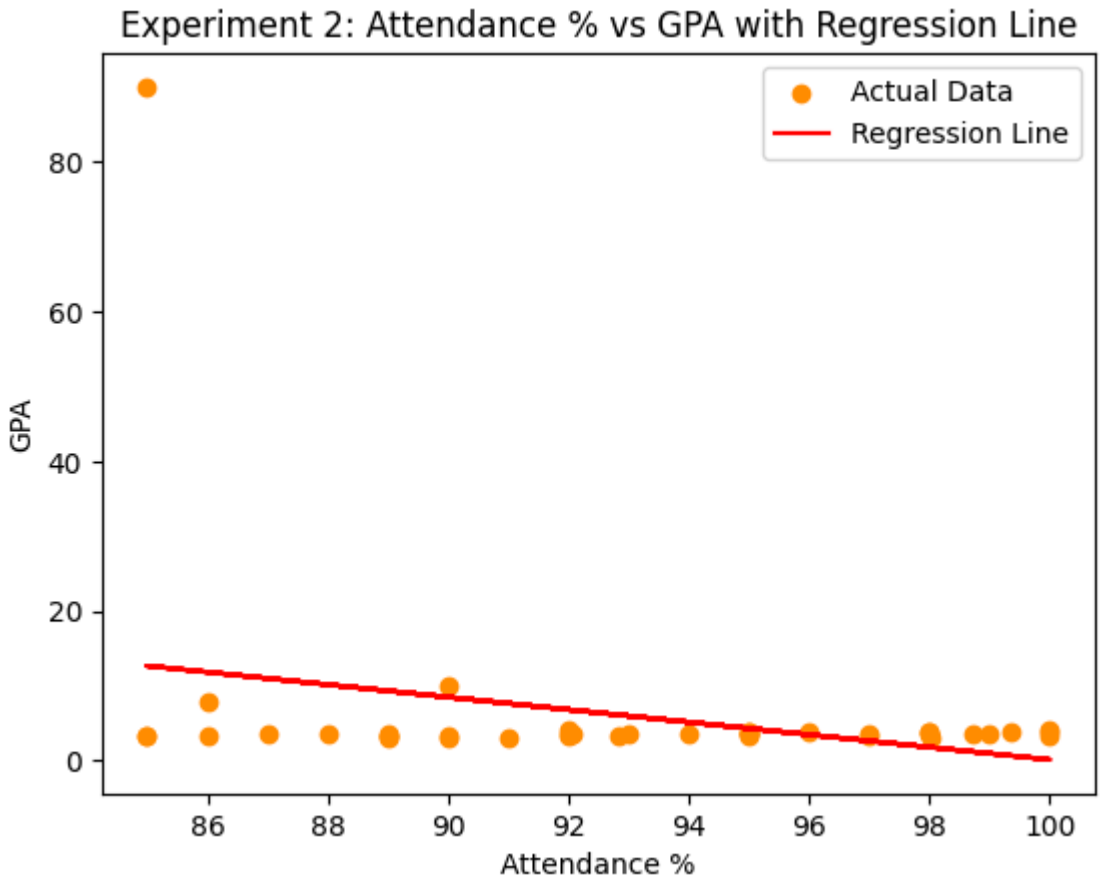
```
model2 = LinearRegression()
model2.fit(X2, y2)

# Predict GPA using the model
y2_pred = model2.predict(X2)

# Results
slope2     = model2.coef_[0]
intercept2 = model2.intercept_
r2_2       = r2_score(y2, y2_pred)
mse2       = mean_squared_error(y2, y2_pred)

print("=== Experiment 2 Results ===")
print(f"Slope     : {slope2:.4f}")
print(f"Intercept : {intercept2:.4f}")
print(f"R2 Score  : {r2_2:.4f}")
print(f"MSE       : {mse2:.4f}")
print()
print("Interpretation:")
print(f"- Slope {slope2:.4f}: For every 1% increase in Attendance, GPA increases by {slope2:.4f} points.")
print(f"- Intercept {intercept2:.4f}: This is the predicted GPA when Attendance is 0 (baseline value).")
print(f"- R2 Score {r2_2:.4f}: The model explains {r2_2*100:.2f}% of the variation in GPA.")
print(f"- MSE {mse2:.4f}: The average squared difference between actual and predicted GPA is {mse2:.4f}.")
```

```
=== Experiment 2 Results ===
Slope     : -0.8363
Intercept : 83.7276
R2 Score  : 0.0830
MSE       : 167.5555

Interpretation:
- Slope -0.8363: For every 1% increase in Attendance, GPA increases by -0.8363 points.
- Intercept 83.7276: This is the predicted GPA when Attendance is 0 (baseline value).
- R2 Score 0.0830: The model explains 8.30% of the variation in GPA.
- MSE 167.5555: The average squared difference between actual and predicted GPA is 167.5555.
```

In [106…

```
# Plot regression line over scatter plot
plt.scatter(data2['Attendance'], y2, color='darkorange', label='Actual Data')
plt.plot(data2['Attendance'], y2_pred, color='red', label='Regression Line')
plt.xlabel('Attendance %')
```

```
plt.ylabel('GPA')
plt.title('Experiment 2: Attendance % vs GPA with Regression Line')
plt.legend()
plt.show()
```




---

# Part B: Simple Linear Regression using Scikit-learn

## Experiment 1: CIA % → GPA

In [107…
```
# Step 1: Split the dataset into training and testing sets
data1 = df[['CIA', 'GPA']].dropna()

X1 = data1[['CIA']]
y1 = data1['GPA']

X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size=0.2, random_state=42)

print("Total samples :", len(X1))
print("Training size :", len(X1_train), "(80%)")
print("Testing size  :", len(X1_test),  "(20%)")
```

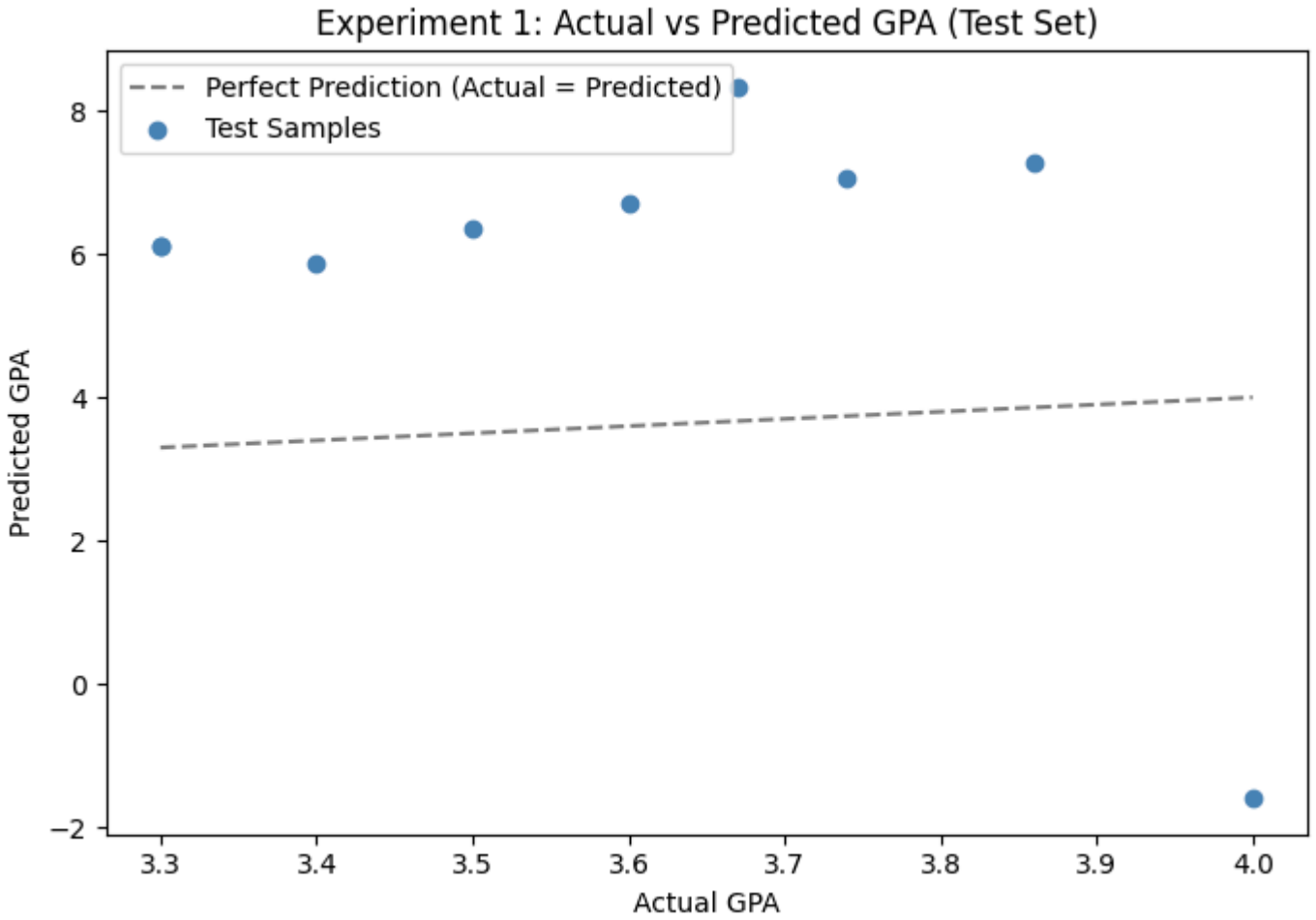
```
Total samples : 43
Training size : 34 (80%)
Testing size  : 9 (20%)
```

In [108…
```
# Step 2: Train the model using LinearRegression
model_b1 = LinearRegression()
model_b1.fit(X1_train, y1_train)

# Step 3: Obtain Slope and Intercept
print("Slope     :", round(model_b1.coef_[0], 4))
print("Intercept :", round(model_b1.intercept_, 4))
print()
print("Regression Equation: GPA =", round(model_b1.coef_[0], 4), "* CIA +", round(model_b1.intercept_, 4))
```

```
Slope     : 0.1169
Intercept : -2.067

Regression Equation: GPA = 0.1169 * CIA + -2.067
```

In [109…
```
# Step 4: Predict output values using the trained model
y1_pred_test = model_b1.predict(X1_test)

# Actual vs Predicted scatter plot
x_line = np.linspace(y1_test.min(), y1_test.max(), 100)

plt.figure(figsize=(7, 5))
plt.plot(x_line, x_line, color='gray', linestyle='--', label='Perfect Prediction (Actual = Predicted)')
plt.scatter(y1_test, y1_pred_test, color='steelblue', zorder=3, label='Test Samples')
plt.xlabel('Actual GPA')
```

```
plt.ylabel('Predicted GPA')
plt.title('Experiment 1: Actual vs Predicted GPA (Test Set)')
plt.legend()
plt.tight_layout()
plt.show()

print("Each dot is one test student.")
print("Dots close to the dashed line = accurate prediction.")
print("Dots far from the line = larger prediction error.")
```



```
Each dot is one test student.
Dots close to the dashed line = accurate prediction.
Dots far from the line = larger prediction error.
```

## Experiment 2: Attendance % → GPA

In [110…

```python
# Step 1: Split the dataset into training and testing sets
data2 = df[['Attendance', 'GPA']].dropna()

X2 = data2[['Attendance']]
y2 = data2['GPA']

X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.2, random_state=42)

print("Total samples :", len(X2))
print("Training size :", len(X2_train), "(80%)")
print("Testing size  :", len(X2_test),  "(20%)")
```

```
Total samples : 40
Training size : 32 (80%)
Testing size  : 8 (20%)
```

In [111…

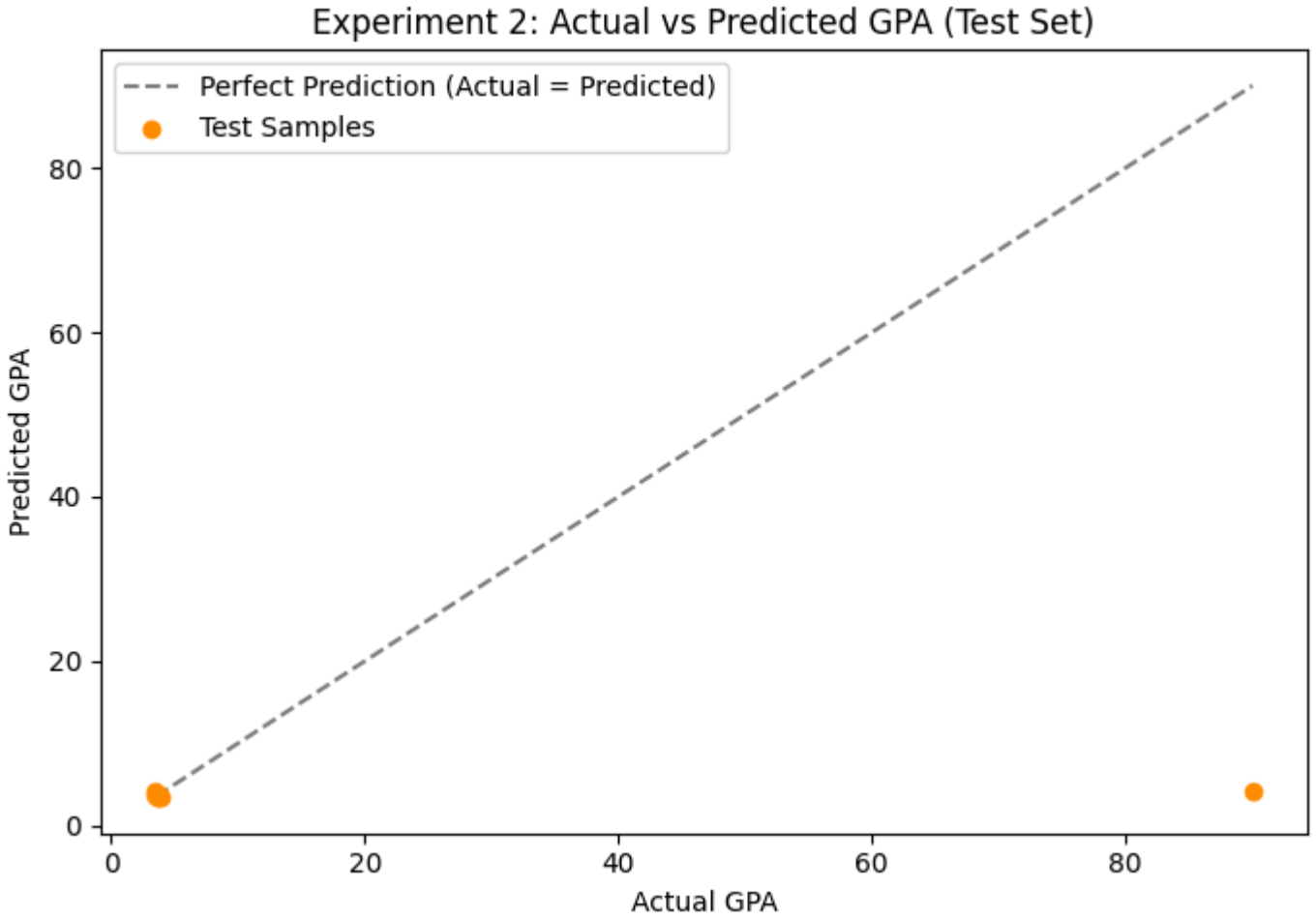
```python
# Step 2: Train the model using LinearRegression
model_b2 = LinearRegression()
model_b2.fit(X2_train, y2_train)

# Step 3: Obtain Slope and Intercept
print("Slope     :", round(model_b2.coef_[0], 4))
print("Intercept :", round(model_b2.intercept_, 4))
print()
print("Regression Equation: GPA =", round(model_b2.coef_[0], 4), "* Attendance +", round(model_b2.intercept_, 4))
```

```
Slope     : -0.0533
Intercept : 8.7389

Regression Equation: GPA = -0.0533 * Attendance + 8.7389
```

In [112…

```python
# Step 4: Predict output values using the trained model
y2_pred = model_b2.predict(X2_test)
```

```
# Actual vs Predicted scatter plot
x_line = np.linspace(y2_test.min(), y2_test.max(), 100)

plt.figure(figsize=(7, 5))
plt.plot(x_line, x_line, color='gray', linestyle='--', label='Perfect Prediction (Actual = Predicted)')
plt.scatter(y2_test, y2_pred, color='darkorange', zorder=3, label='Test Samples')
plt.xlabel('Actual GPA')
plt.ylabel('Predicted GPA')
plt.title('Experiment 2: Actual vs Predicted GPA (Test Set)')
plt.legend()
plt.tight_layout()
plt.show()

print("Each dot is one test student.")
print("Dots close to the dashed line = accurate prediction.")
print("Dots far from the line = larger prediction error.")
```




```
Each dot is one test student.
Dots close to the dashed line = accurate prediction.
Dots far from the line = larger prediction error.
```

---

## Part C: Manual Computation using Ordinary Least Squares (OLS)

The same training data from Part B is used so the comparison is fair.

## Experiment 1: CIA % → GPA

In [113…

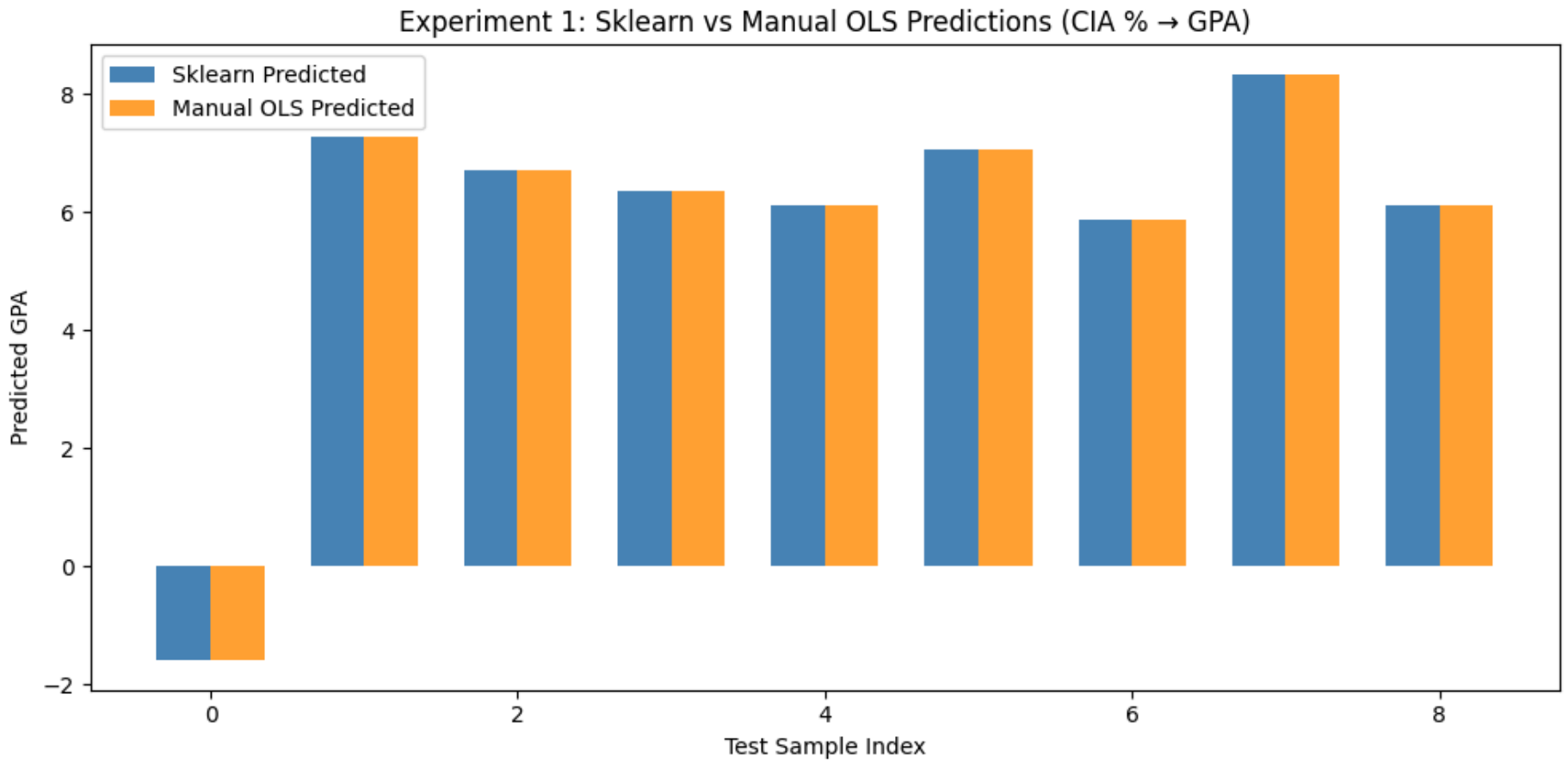
```
# Manual OLS computation using training data (same data Part B used)
x1_vals = X1_train['CIA'].values
y1_vals = y1_train.values

x1_mean = np.mean(x1_vals)
y1_mean = np.mean(y1_vals)

# OLS Slope formula: m = sum((x - x_mean)(y - y_mean)) / sum((x - x_mean)^2)
slope_ols1 = np.sum((x1_vals - x1_mean) * (y1_vals - y1_mean)) / np.sum((x1_vals - x1_mean)**2)

# OLS Intercept formula: b = y_mean - m * x_mean
intercept_ols1 = y1_mean - slope_ols1 * x1_mean

print("Manual OLS Results - Experiment 1:")
print("Mean of CIA (x̄):", round(x1_mean, 4))
print("Mean of GPA (ȳ) :", round(y1_mean, 4))
print()
print("Slope     :", round(slope_ols1, 4))
print("Intercept :", round(intercept_ols1, 4))
print()
print("Regression Equation: GPA =", round(slope_ols1, 4), "* CIA +", round(intercept_ols1, 4))
```

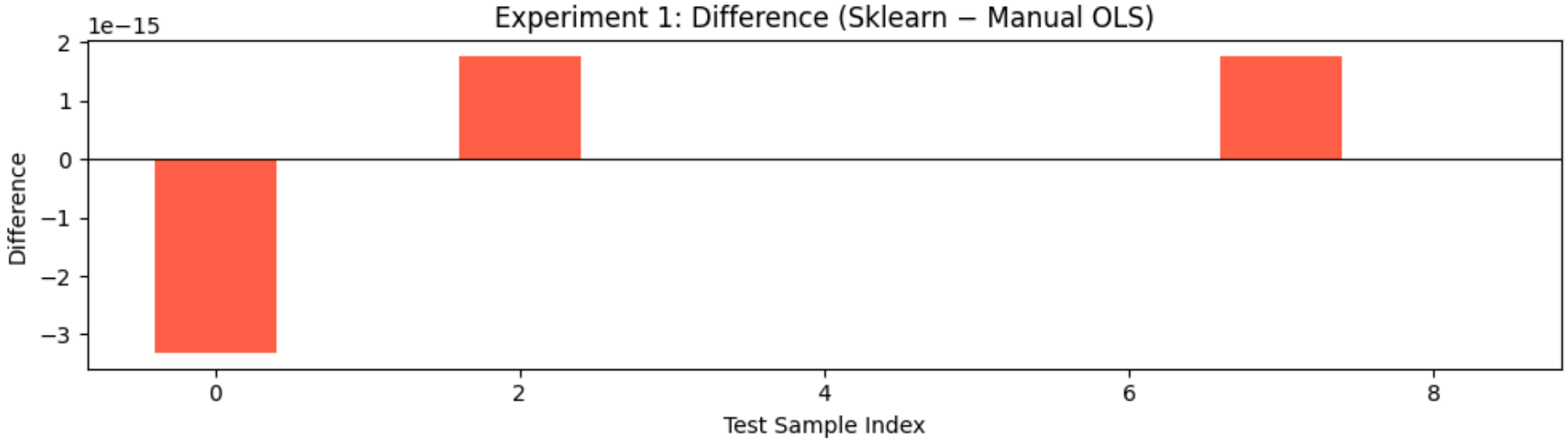
```
Manual OLS Results - Experiment 1:
Mean of CIA (x̄): 71.5897
Mean of GPA (ȳ) : 6.3021

Slope     : 0.1169
Intercept : -2.067

Regression Equation: GPA = 0.1169 * CIA + -2.067
```

In [114…

```
# Predict using manual OLS equation on the same test set
y1_ols_pred = slope_ols1 * X1_test['CIA'].values + intercept_ols1

indices = np.arange(len(X1_test))
width = 0.35
```

```
# Chart 1: Side-by-side bar chart of both predictions
plt.figure(figsize=(10, 5))
plt.bar(indices - width/2, y1_pred_test, width, label='Sklearn Predicted', color='steelblue')
plt.bar(indices + width/2, y1_ols_pred,  width, label='Manual OLS Predicted', color='darkorange', alpha=0.8)
plt.xlabel('Test Sample Index')
plt.ylabel('Predicted GPA')
plt.title('Experiment 1: Sklearn vs Manual OLS Predictions (CIA % → GPA)')
plt.legend()
plt.tight_layout()
plt.show()

# Chart 2: Difference between both methods
diff1 = y1_pred_test - y1_ols_pred
plt.figure(figsize=(10, 3))
plt.bar(indices, diff1, color='tomato')
plt.axhline(0, color='black', linewidth=0.8)
plt.xlabel('Test Sample Index')
plt.ylabel('Difference')
plt.title('Experiment 1: Difference (Sklearn − Manual OLS)')
plt.tight_layout()
plt.show()

print("Both bars in the first chart overlap almost perfectly.")
print("The difference chart shows values at or extremely close to 0.")
print("This confirms both methods produce the same predictions.")
```


Experiment 1: Sklearn vs Manual OLS Predictions (CIA % → GPA)
Sklearn Predicted
Manual OLS Predicted
Predicted GPA
Test Sample Index
8
6
4
2
0
−2
0
2
4
6
8




Both bars in the first chart overlap almost perfectly.
The difference chart shows values at or extremely close to 0.
This confirms both methods produce the same predictions.

## Experiment 2: Attendance % → GPA

In [115…

```
# Manual OLS computation using training data (same data Part B used)
x2_vals = X2_train['Attendance'].values
y2_vals = y2_train.values

x2_mean = np.mean(x2_vals)
y2_mean = np.mean(y2_vals)

# OLS Slope formula: m = sum((x - x_mean)(y - y_mean)) / sum((x - x_mean)^2)
slope_ols2 = np.sum((x2_vals - x2_mean) * (y2_vals - y2_mean)) / np.sum((x2_vals - x2_mean)**2)

# OLS Intercept formula: b = y_mean - m * x_mean
intercept_ols2 = y2_mean - slope_ols2 * x2_mean

print("Manual OLS Results - Experiment 2:")
print("Mean of Attendance (x̄):", round(x2_mean, 4))
print("Mean of GPA (ȳ)        :", round(y2_mean, 4))
print()
print("Slope     :", round(slope_ols2, 4))
```

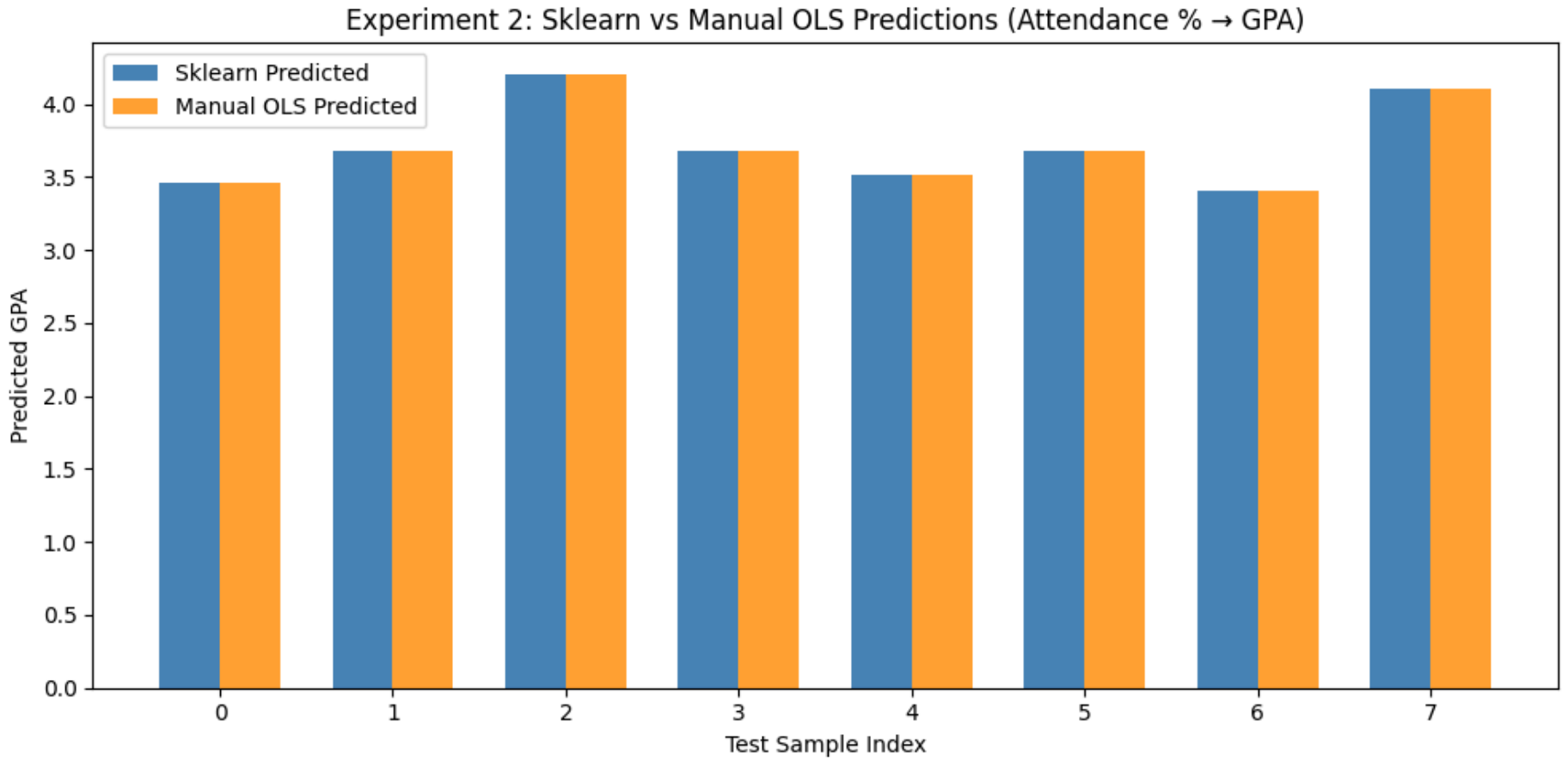
```
print("Intercept :", round(intercept_ols2, 4))
print()
print("Regression Equation: GPA =", round(slope_ols2, 4), "* Attendance +", round(intercept_ols2, 4))
```

```
Manual OLS Results - Experiment 2:
Mean of Attendance (x̄): 92.7181
Mean of GPA (ȳ)        : 3.7972

Slope     : -0.0533
Intercept : 8.7389

Regression Equation: GPA = -0.0533 * Attendance + 8.7389
```

In [116…

```
# Predict using manual OLS equation on the same test set
y2_ols_pred = slope_ols2 * X2_test['Attendance'].values + intercept_ols2

indices = np.arange(len(X2_test))
width = 0.35

# Chart 1: Side-by-side bar chart of both predictions
plt.figure(figsize=(10, 5))
plt.bar(indices - width/2, y2_pred,     width, label='Sklearn Predicted', color='steelblue')
plt.bar(indices + width/2, y2_ols_pred, width, label='Manual OLS Predicted', color='darkorange', alpha=0.8)
plt.xlabel('Test Sample Index')
plt.ylabel('Predicted GPA')
plt.title('Experiment 2: Sklearn vs Manual OLS Predictions (Attendance % → GPA)')
plt.legend()
plt.tight_layout()
plt.show()

# Chart 2: Difference between both methods
diff2 = y2_pred - y2_ols_pred
plt.figure(figsize=(10, 3))
plt.bar(indices, diff2, color='tomato')
plt.axhline(0, color='black', linewidth=0.8)
plt.xlabel('Test Sample Index')
plt.ylabel('Difference')
plt.title('Experiment 2: Difference (Sklearn − Manual OLS)')
plt.tight_layout()
plt.show()
```

```
print("Both bars in the first chart overlap almost perfectly.")
print("The difference chart shows values at or extremely close to 0.")
print("This confirms both methods produce the same predictions.")
```




```
Both bars in the first chart overlap almost perfectly.
The difference chart shows values at or extremely close to 0.
This confirms both methods produce the same predictions.
```

## Observations

- **Both methods give the same results.** Scikit-learn's `LinearRegression` uses OLS internally, so the slope and intercept are computed using the same formula.
- **The Difference column shows 0 or an extremely small number** (like 0.000000). Any tiny difference is due to floating-point precision in the computer — not a real difference in the formula.
- **Conclusion:** Manual OLS and Scikit-learn produce identical predictions for simple linear regression. Scikit-learn simply automates the OLS computation.

---

# Parameter Saving Task

Save the slope and intercept learned by both models into a Pickle file ( `linear_regression_weights.pkl` ), then load and use them for prediction.

In [117…

```python
import pickle

# 1. Save slope and intercept for both experiments into a pickle file
params = {
    'experiment_1': {
        'variable'  : 'CIA %',
        'slope'     : model_b1.coef_[0],
        'intercept' : model_b1.intercept_
    },
    'experiment_2': {
        'variable'  : 'Attendance %',
        'slope'     : model_b2.coef_[0],
        'intercept' : model_b2.intercept_
    }
}

with open('linear_regression_weights.pkl', 'wb') as f:
    pickle.dump(params, f)

print("Parameters saved to linear_regression_weights.pkl")
print()
print("Saved values:")
for exp, values in params.items():
    print(f"\n{exp}  ({values['variable']} -> GPA):")
    print(f"  Slope     : {values['slope']:.4f}")
    print(f"  Intercept : {values['intercept']:.4f}")
```

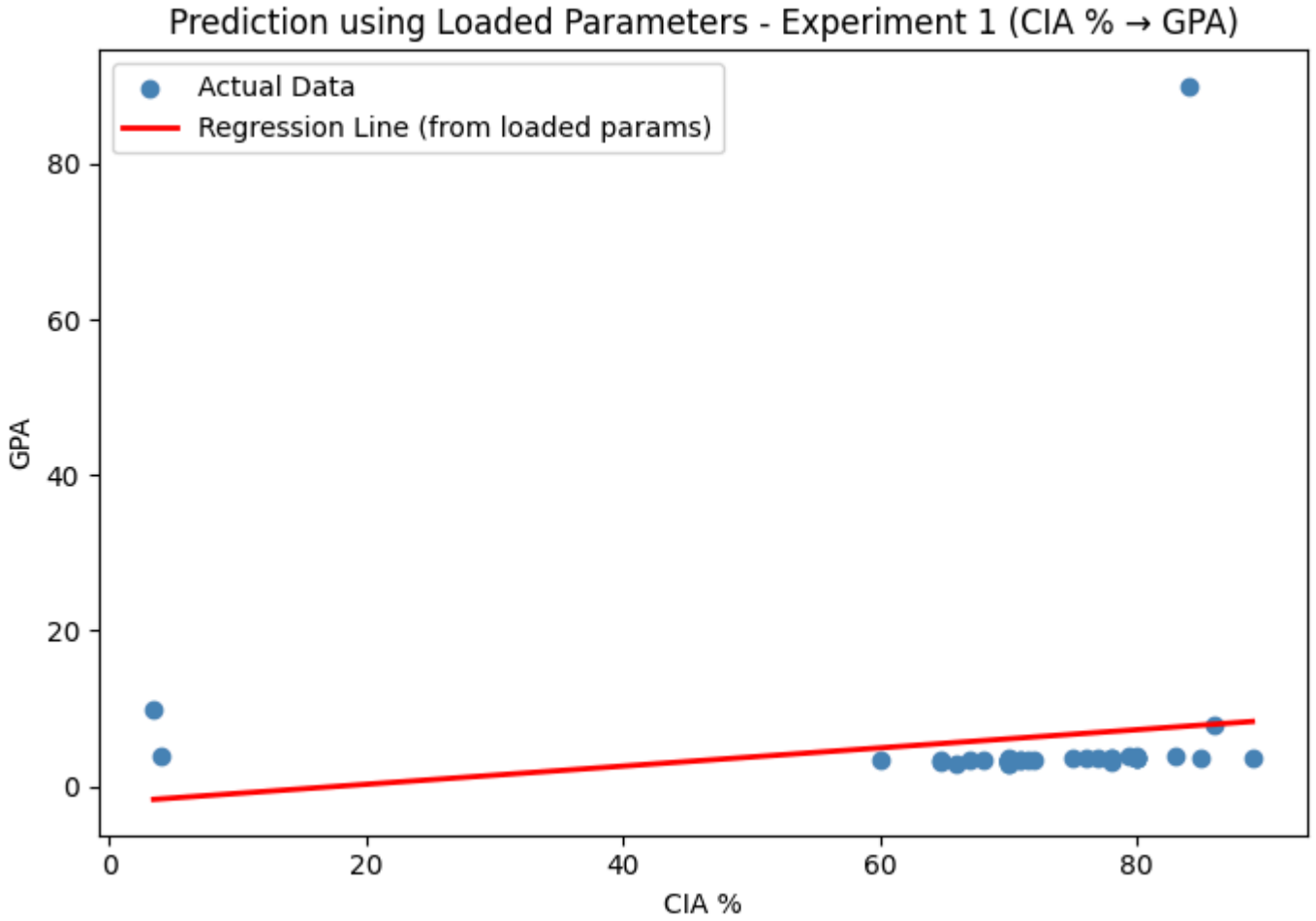
```
Parameters saved to linear_regression_weights.pkl

Saved values:

experiment_1  (CIA % -> GPA):
  Slope     : 0.1169
  Intercept : -2.0670

experiment_2  (Attendance % -> GPA):
  Slope     : -0.0533
  Intercept : 8.7389
```

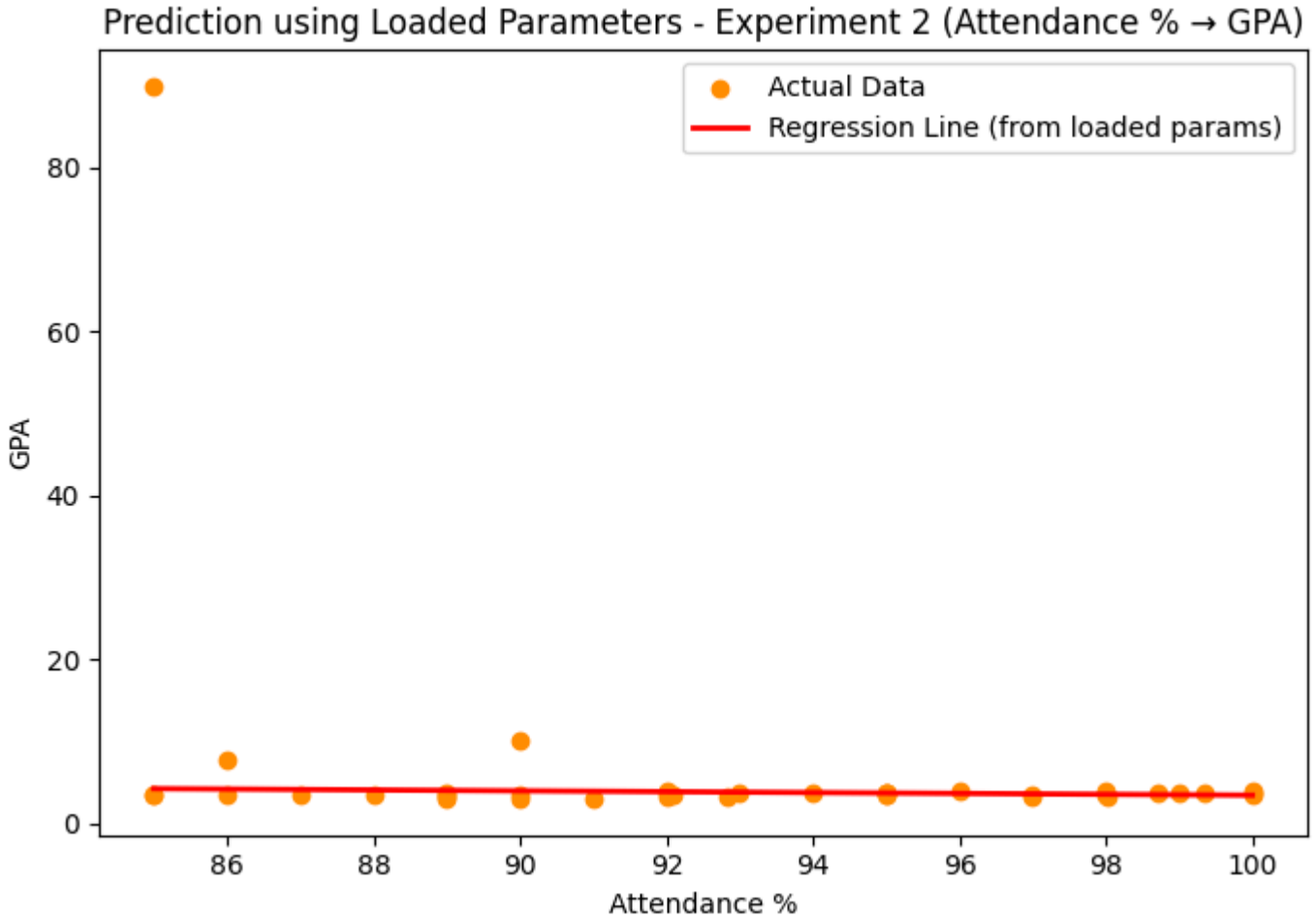
In [118…
```
# 2. Load the parameters from the pickle file
with open('linear_regression_weights.pkl', 'rb') as f:
    loaded_params = pickle.load(f)

print("Parameters loaded successfully from linear_regression_weights.pkl")
print()
for exp, values in loaded_params.items():
    print(f"{exp}  ({values['variable']} -> GPA):")
    print(f"  Slope     : {values['slope']:.4f}")
    print(f"  Intercept : {values['intercept']:.4f}")
    print()
```

```
Parameters loaded successfully from linear_regression_weights.pkl

experiment_1  (CIA % -> GPA):
  Slope     : 0.1169
  Intercept : -2.0670

experiment_2  (Attendance % -> GPA):
  Slope     : -0.0533
  Intercept : 8.7389
```

In [119…
```
# 3. Use loaded parameters to predict GPA for a range of values and plot the regression lines

slope_c1     = loaded_params['experiment_1']['slope']
intercept_c1 = loaded_params['experiment_1']['intercept']
slope_c2     = loaded_params['experiment_2']['slope']
intercept_c2 = loaded_params['experiment_2']['intercept']

# --- Experiment 1: CIA % → GPA ---
cia_range  = np.linspace(df['CIA'].dropna().min(), df['CIA'].dropna().max(), 100)
gpa_pred_c1 = slope_c1 * cia_range + intercept_c1

plt.figure(figsize=(7, 5))
plt.scatter(df['CIA'], df['GPA'], color='steelblue', label='Actual Data')
plt.plot(cia_range, gpa_pred_c1, color='red', linewidth=2, label='Regression Line (from loaded params)')
plt.xlabel('CIA %')
plt.ylabel('GPA')
plt.title('Prediction using Loaded Parameters - Experiment 1 (CIA % → GPA)')
```

```
plt.legend()
plt.tight_layout()
plt.show()

# --- Experiment 2: Attendance % → GPA ---
att_range   = np.linspace(df['Attendance'].dropna().min(), df['Attendance'].dropna().max(), 100)
gpa_pred_c2 = slope_c2 * att_range + intercept_c2

plt.figure(figsize=(7, 5))
plt.scatter(df['Attendance'], df['GPA'], color='darkorange', label='Actual Data')
plt.plot(att_range, gpa_pred_c2, color='red', linewidth=2, label='Regression Line (from loaded params)')
plt.xlabel('Attendance %')
plt.ylabel('GPA')
plt.title('Prediction using Loaded Parameters - Experiment 2 (Attendance % → GPA)')
plt.legend()
plt.tight_layout()
plt.show()

print("Regression lines above are drawn using only the slope and intercept loaded from the pickle file.")
print("No model object was needed — just the two numbers are enough to make predictions.")
```


Prediction using Loaded Parameters - Experiment 1 (CIA % → GPA)
Actual Data
Regression Line (from loaded params)
GPA
CIA %
0
20
40
60
80




Regression lines above are drawn using only the slope and intercept loaded from the pickle file.
No model object was needed — just the two numbers are enough to make predictions.